Инструкция по запуску + обзор на код

Содержание

1 Сбор данных **2**
- 1.1 SQL-подготовка данных . . . 2

2 Устройство MLflow **2**
- 2.1 Иерархия запусков . . . 3
- 2.2 Уровень канала . . . 3
- 2.3 Уровень модели . . . 3
- 2.4 Уровень Optuna trial . . . 3

3 Разработка **3**
- 3.1 Общая организация кода . . . 3
- 3.2 Организация конфигураций . . . 4
- 3.3 Роль params.yaml . . . 4
- 3.4 Связь между этапами . . . 5
- 3.5 Загрузка данных: load . . . 5
  - 3.5.1 Запуск этапа . . . 6
- 3.6 Обработка данных: process . . . 6
  - 3.6.1 Конфигурация этапа . . . 6
  - 3.6.2 Логика обработки . . . 7
  - 3.6.3 Обработка train, oot и oos . . . 7
  - 3.6.4 Результаты этапа . . . 7
  - 3.6.5 Запуск этапа . . . 8
- 3.7 Профилирование данных: profiling . . . 8
  - 3.7.1 Запуск этапа . . . 8
- 3.8 Обучение модели: train . . . 9
  - 3.8.1 Конфигурация этапа . . . 9
  - 3.8.2 Компоненты обучения . . . 9
  - 3.8.3 Финальное обучение и результаты . . . 10
  - 3.8.4 Запуск этапа . . . 10

4 Описание ноутбуков **11**
- 4.1 Параметры запуска . . . 11
- 4.2 Загрузка данных: load.ipynb . . . 11
- 4.3 Обработка данных: process.ipynb . . . 11
- 4.4 Профилирование: profiler.ipynb . . . 12
- 4.5 Обучение: train.ipynb . . . 12
- 4.6 Связь с основными этапами . . . 12

1 Сбор данных

1.1 SQL-подготовка данных

Скрипты сбора расположены в каталоге `sql/` и организованы по единому шаблону для всех направлений: `CC`, `DEP`, `DEP_OUT`, `IB`, `MB` и `PUSH`. Различаются условия отбора контактов и правила расчёта таргета.

Параметры сбора задаются константами в начале SQL-файла:

- `COLLECT_DATA_FROM`, `COLLECT_DATA_TO` — границы периода сбора;
- `TEST_DATA_FROM` — дата начала тестового периода;
- `OOS_SAMPLE_P` — доля OOS-выборки, установленная в `0.15` (15%).

Сбор выполняется последовательно:

1. Отбираются кампании за заданный период.
2. Формируется `BASE` — база подходящих наблюдений после фильтрации контактов и исключения неоднозначных записей. Она хранит идентификаторы клиента и кампании, а также дату наблюдения.
3. Формируется `TARGET` — результат взаимодействия, рассчитанный по правилам соответствующего направления. Таблица содержит ключи наблюдения и бинарный таргет.
4. Для клиентов из `BASE` рассчитываются признаки на дату наблюдения: характеристики клиента, активные продукты, история событий и клиентские метрики.
5. Признаки и таргет присоединяются к базе наблюдений, образуя итоговую таблицу для дальнейшей выгрузки.

Итоговые таблицы следуют общему шаблону: идентификатор клиента, дата наблюдения, набор признаков и таргет соответствующего направления.

На выходе для каждого направления формируются три таблицы с одинаковой структурой:

- `TRAIN` — обучающая выборка;
- `TEST` — выборка за тестовый период;
- `OOS` — отдельная выборка для проверки качества модели.

2 Устройство MLflow

MLflow используется для хранения результатов экспериментов: параметров предобработки, настроек моделей, метрик качества и обученных моделей. Все запуски проекта находятся в эксперименте `retail/chanels/models`.

2.1 Иерархия запусков

Запуски организованы в три уровня:

1. Канал — отдельный родительский run для каждого из шести каналов.
2. Модель — три вложенных run внутри каждого канала: для CatBoost, XGBoost и LightGBM.
3. Попытка Optuna — десять вложенных run внутри запуска каждой модели, по одному на каждый trial.

Такая структура позволяет сравнивать модели в рамках одного канала, а внутри модели — просматривать результаты отдельных попыток подбора гиперпараметров.

2.2 Уровень канала

Родительский run объединяет результаты работы с данными соответствующего канала и запуски его моделей.

В разделе `Artifacts`, в каталоге `processed`, хранятся настройки и параметры препроцессора. Они описывают обработку данных, использованную при обучении моделей этого канала.

2.3 Уровень модели

Для каждого канала созданы отдельные вложенные run для CatBoost, XGBoost и LightGBM. В них сохраняются метрики качества и параметры обученных моделей.

В разделе `Artifacts` хранится обёртка обученной модели в формате `.joblib`. Этот артефакт используется для последующей загрузки модели и получения предсказаний.

2.4 Уровень Optuna trial

Внутри запуска каждой модели находятся десять вложенных run, соответствующих попыткам подбора гиперпараметров Optuna.

В каждой попытке модель оценивается на временных разбиениях с помощью `TimeSeriesSplit`. В run попытки логируется среднее значение `ROC AUC` по разбиениям. По этой метрике сравниваются варианты гиперпараметров и выбирается лучший результат.

3 Разработка

3.1 Общая организация кода

Код проекта разделён на этапы: загрузка данных, обработка, профилирование и обучение модели. Точки входа этапов расположены в каталоге `stages/`, а реализация используемых компонентов — в `src/`.

Сценарий каждого этапа читает параметры запуска и конфигурации, создаёт необходимые компоненты, выполняет обработку и сохраняет результаты. Настройки компонентов отделены от кода: конкретный вариант работы этапа определяется выбранной конфигурацией.

3.2 Организация конфигураций

Основной каталог конфигураций проекта — `conf/`. Он содержит общие настройки проекта и настройки отдельных этапов.

К общим, относительно статическим настройкам относятся:

- `conf/secrets/` — конфигурации доступов к базе данных и MLflow;
- `conf/common/paths.yaml` — пути к данным, конфигурациям, моделям, манифестам и другим артефактам.

Настройки отдельных этапов расположены в `conf/stages/`. Каждый подкаталог `stage_X/` описывает параметры соответствующего этапа, доступные конфигурации и способ его запуска.

Внутри каталога этапа используется следующая структура:

- `config/` — конфигурации конкретных вариантов работы этапа: описание источника данных, настройки преобразований, профилирования, модели или валидации;
- `manifest/` — манифесты с метаинформацией о выполнении этапа и полученных результатах;
- `dvc.yaml` — описание запуска через `dvc repro`: команды, зависимости, отслеживаемые параметры и выходные файлы;
- YAML-файл этапа, например `load.yaml` или `process.yaml`, — структура параметров и значения по умолчанию, с которыми работает Hydra. При запуске эти значения могут переопределяться аргументами команды.

3.3 Роль params.yaml

Файл `params.yaml` — главный конфигуратор запуска эксперимента. Он определяет:

- имена конфигураций из каталогов `conf/stages/stage_X/config/`, которые должны использовать этапы;
- имена промежуточных и итоговых артефактов;
- общие параметры эксперимента, включая имена эксперимента и запуска в MLflow.

В `params.yaml` выбираются конфигурации, но не дублируется их содержимое. Подробные настройки источников, преобразований и обучения остаются в соответствующих конфигурационных файлах.

При запуске значения из `params.yaml` подставляются в команды из `dvc.yaml` и передаются Python-модулям через Hydra. Код этапа получает параметры, определяет пути к выбранным конфигурациям, читает их и запускает компоненты с указанными настройками.

Таким образом, `params.yaml` задаёт вариант эксперимента, `dvc.yaml` описывает выполнение этапов, а YAML-файл Hydra определяет структуру параметров, которые получает код.

3.4 Связь между этапами

Этапы обмениваются результатами через файлы данных и манифесты из каталогов `manifest/`. Манифест хранит метаинформацию о результате этапа: пути к артефактам, идентификатор MLflow run и дополнительные сведения для последующей обработки.

Например, после загрузки сохраняется путь к исходной выборке, а после обработки — путь к подготовленным данным и описание их столбцов. Следующий этап читает эти сведения из манифеста и использует соответствующие артефакты.

Имена результатов согласуются через `params.yaml`, зависимости между этапами описываются в `dvc.yaml`, а сведения о полученных результатах передаются через манифесты.

3.5 Загрузка данных: load

Этап `load` выгружает подготовленные таблицы из Teradata в локальные CSV-файлы. Точка входа — `stages/load.py`, реализация выгрузки — функция `load_frame` в `src/load_tools/tools.py`. SQL-скрипты на этом этапе не выполняются: исходные таблицы должны быть подготовлены заранее.

В `conf/stages/stage_load/dvc.yaml` описаны три стадии: `load_train`, `load_oot` и `load_oos`. Все они запускают один модуль `stages.load`, но передают разные конфигурации источника и имена выходных файлов.

Для каждой выборки в `params.yaml` задаются:

- `raw_config` — имя конфигурации источника. В команду запуска передаётся как `config_name`;
- `raw_name` — имя результата выгрузки, используемое для CSV-файла и манифеста.

Работа этапа выполняется в следующем порядке:

1. Загружаются общие пути, параметры доступа к Teradata и настройки MLflow.
2. Конфигурация источника читается в объект `SourceTable`. По параметрам эксперимента определяется MLflow run.
3. Функция `load_frame` открывает соединение с Teradata и читает таблицу запросом `SELECT * FROM <table.full_name>`. Результат выгружается в CSV через `fastexport`. После выполнения соединение закрывается.
4. Сохраняется манифест с путём к CSV и идентификатором MLflow run. Манифест также передаётся в MLflow.

Результаты каждой стадии:

- `data/raw/<raw_name>.csv` — исходная выборка;
- `conf/stages/stage_load/manifest/<raw_name>_manifest.yaml` — метаинформация для следующего этапа.

Этап не преобразует данные и не разделяет их на выборки: `train`, `oot` и `oos` выгружаются из отдельных настроенных источников. Следующий этап `process` получает путь к данным и идентификатор запуска из манифеста загрузки.

3.5.1 Запуск этапа

Перед запуском необходимо выбрать конфигурации источников и имена результатов в секции `stage_load` файла `params.yaml`, а также проверить настройки доступа к Teradata и MLflow.

Для запуска загрузки всех трёх выборок из корня проекта:

```
dvc repro conf/stages/stage_load/dvc.yaml
```

Для загрузки только обучающей выборки:

```
dvc repro conf/stages/stage_load/dvc.yaml:load_train
```

Поэтапную работу загрузчика можно посмотреть в `notebooks/load.ipynb`. В нём отдельно выполняются сборка конфигурации Hydra, инициализация настроек, выгрузка данных, сохранение манифеста и его передача в MLflow. В конце можно просмотреть первые строки полученного CSV.

Параметры источника и имени результата в ноутбуке задаются непосредственно в его ячейках.

3.6 Обработка данных: process

Этап `process` подготавливает исходные CSV-файлы для обучения и оценки модели. Точка входа — `stages/process.py`, реализация преобразований — `src/process_tools/transformers.py`.

В `conf/stages/stage_process/dvc.yaml` описаны три стадии: `process_train`, `process_oot` и `process_oos`. Они используют один модуль `stages.process`, но отличаются входными данными, именами результатов и режимом работы преобразований.

3.6.1 Конфигурация этапа

В секции `stage_process` файла `params.yaml` задаются:

- `raw_data_desc_name` — имя конфигурации с описанием столбцов исходных данных;
- `process_config` — имя конфигурации преобразований. В команду запуска передаётся как `config_name`;
- `preprocessor_state_name` — имя файла состояния преобразований;
- `process_name` для каждой выборки — имя обработанного CSV и соответствующего манифеста.

Конфигурация исходных данных читается в `RawDataDesc` и определяет признаки, таргет, временной столбец и идентификаторы. Конфигурация преобразований читается в `ProcessDescription` и задаёт список компонентов, порядок их применения и параметры.

3.6.2 Логика обработки

Работа этапа выполняется в следующем порядке:

1. Загружаются общие пути, настройки MLflow и конфигурации обработки.
2. Из манифеста загрузки читаются путь к исходному CSV и идентификатор MLflow run. Данные загружаются в `pandas.DataFrame`.
3. Создаётся `MainTransformer`, который собирает последовательность преобразований в `sklearn.pipeline.Pipeline`. Компоненты выбираются по именам через реестр `BaseTransformer.registry`.
4. Выполняется обучение преобразований или загрузка ранее сохранённого состояния, после чего данные преобразуются.
5. Формируется описание обработанных столбцов, сохраняются CSV и манифест. Манифест и состояние преобразований передаются в MLflow.

В конфигурации `processor_desc` преобразования выполняются в следующем порядке:

1. `ColumnSelector` — оставляет столбцы, указанные в описании исходных данных;
2. `CategoricalEncoder` — кодирует выбранные категориальные признаки целыми числами и присваивает столбцам тип `category`. Пропуски и неизвестные категории получают код `-1`;
3. `TimeSorter` — сортирует строки по временному столбцу, указанному в конфигурации.

3.6.3 Обработка train, oot и oos

Для `train` используется режим `fit=true`. Параметры компонентов берутся из конфигурации, преобразования обучаются на исходной выборке, а их состояние сохраняется в файл `joblib`. Затем выполняется `transform`.

Для `oot` и `oos` используется `fit=false`. Компоненты создаются с параметрами из сохранённого состояния и выполняют только `transform`, без повторного обучения. Это обеспечивает одинаковые правила обработки и кодирования категорий для всех выборок.

Стадии `process_oot` и `process_oos` зависят от файла состояния, полученного в `process_train`.

3.6.4 Результаты этапа

Для каждой выборки сохраняются:

- `data/processed/<process_name>.csv` — обработанные данные;
- `conf/stages/stage_process/manifest/<process_name>_manifest.yaml` — манифест обработки.

При обработке `train` дополнительно сохраняется состояние преобразований: `conf/stages/stage_process/manifest/<preprocessor_state_name>.joblib`.

Манифест содержит пути к данным и состоянию преобразований, идентификатор MLflow run и описание обработанной выборки: числовые и категориальные признаки, таргет, временной столбец и идентификаторы. Это описание используется на последующих этапах.

3.6.5 Запуск этапа

Перед запуском необходимо выбрать конфигурации и имена результатов в `params.yaml`. Исходные CSV и манифесты формируются этапом `load`.

Для запуска обработки всех трёх выборок из корня проекта:

```
dvc repro conf/stages/stage_process/dvc.yaml
```

Для обработки только обучающей выборки:

```
dvc repro conf/stages/stage_process/dvc.yaml:process_train
```

Поэтапную работу можно посмотреть в `notebooks/process.ipynb`. В нём отдельно выполняются сборка конфигурации Hydra, чтение исходных данных, создание и обучение преобразований, применение обработки и сохранение результатов.

Параметры запуска в ноутбуке задаются непосредственно в его ячейках.

3.7 Профилирование данных: profiling

Этап `profiling` строит отчёты по обработанным данным. Точка входа — `stages/profiling.py`, компоненты профилирования расположены в `src/profiling/`.

В текущей DVC-конфигурации описана стадия `profile_train`, которая использует результат обработки обучающей выборки. Имя конфигурации профилирования задаётся параметром `stage_profiling.config` в `params.yaml`.

Работа этапа:

1. Из манифеста обработки читаются путь к CSV, описание столбцов и идентификатор MLflow run.
2. Загружаются данные и конфигурация, определяющая список проверок и их параметры.
3. Компонент `MainProfiler` запускает выбранные проверки.
4. Отчёты сохраняются в каталог, заданный в конфигурации путей, и передаются в MLflow.

Профилирование не изменяет обработанную выборку и не является обязательным входом для обучения модели. В текущем `dvc.yaml` отслеживаемым результатом указан отчёт `psi_features.png`.

3.7.1 Запуск этапа

Для запуска из корня проекта:

```
dvc repro conf/stages/stage_profiling/dvc.yaml
```

3.8 Обучение модели: train

Этап `train` выполняет подбор гиперпараметров, обучение и сохранение модели. Точка входа — `stages/train.py`, основные компоненты расположены в `src/train_tools/`.

На вход подаётся обработанная обучающая выборка. Путь к CSV, списки числовых и категориальных признаков, таргет и идентификатор MLflow run читаются из манифеста этапа `process`.

3.8.1 Конфигурация этапа

В секции `stage_train` файла `params.yaml` задаются:

- `train_config` — имя конфигурации модели и подбора гиперпараметров;
- `validation_config` — имя конфигурации метрик и временной валидации.

Конфигурация обучения определяет:

- `model_name` — класс модели;
- `fixed_params` — фиксированные параметры;
- `search_space` — пространство поиска Optuna;
- `fit_params` — параметры метода обучения;
- `optuna_params` — число попыток, оптимизируемую метрику и направление оптимизации.

Конфигурация валидации задаёт список метрик, их параметры, число временных разбиений `n_splits` и отступ `gap` между обучающей и проверочной частями.

3.8.2 Компоненты обучения

Модели. Модуль `src/train_tools/models.py` задаёт общий интерфейс `BaseModel`. Конкретная модель выбирается по имени через `BaseModel.registry`.

В проекте реализованы обёртки `CatBoostWrapper`, `XGBoostWrapper` и `LightGBMWrapper`. Они приводят работу библиотек к общему интерфейсу: обучение, получение предсказаний и вероятностей, сохранение и загрузка модели.

Метод `clone` создаёт новый, необученный экземпляр обёртки с копией параметров. Он используется, чтобы обучать независимые модели для разных попыток Optuna и разбиений валидации.

Метрики. Компонент `Evaluator` из `src/train_tools/metrics.py` создаёт выбранные метрики через реестр `BaseMetric.registry` и рассчитывает их значения.

Каждая метрика указывает необходимый тип входа: метки классов или вероятности. Функция `predict_inputs` вызывает `predict` и/или `predict_proba` в зависимости от выбранного набора метрик.

Валидация одного разбиения. Компонент `HoldoutValidator` из `src/train_tools/trainers.py` обучает модель на переданной обучающей части. Проверочная часть передаётся в обучение как `eval_set`, после чего на ней рассчитываются предсказания и метрики.

Временная валидация. Компонент `TimeSeriesValidator` формирует разбиения через `TimeSeriesSplit`. Для каждого разбиения создаётся отдельная копия модели и запускается `HoldoutValidator`. Результатом становится таблица метрик по разбиениям.

Разбиение использует порядок строк, а не значения временного столбца. Поэтому данные должны поступать уже отсортированными по времени: в текущей конфигурации это выполняет `TimeSorter` на этапе обработки. Параметр `gap` задаётся количеством строк.

Подбор гиперпараметров. Компонент `OptunaTrainer` управляет поиском параметров. В каждой попытке он:

1. выбирает значения из `search_space`;
2. объединяет их с `fixed_params`;
3. оценивает модель через `TimeSeriesValidator`;
4. возвращает среднее значение выбранной метрики по временным разбиениям.

Для параметров поиска поддерживаются типы `int`, `float` и `categorical`. Лучшая комбинация выбирается по средней метрике с учётом направления оптимизации.

Используемый этапом `MlflowOptunaTrainer` расширяет эту логику сохранением результатов в MLflow. Для модели создаётся дочерний запуск, а для каждой попытки — вложенный запуск с параметрами, средней целевой метрикой и таблицей метрик по разбиениям.

3.8.3 Финальное обучение и результаты

После завершения поиска создаётся новая модель. Её параметры объединяют `fixed_params` и `study.best_params`.

Для финального обучения используется последнее временное разбиение: модель обучается на его обучающей части, а проверочная часть передаётся как `eval_set`. Переобучение на всей выборке в текущем сценарии не выполняется.

После обучения рассчитываются метрики для обучающей и проверочной частей. В MLflow они сохраняются с префиксами `train/` и `eval/`.

Выходные артефакты:

- `models/<train_config>.joblib` — обученная модель;
- `conf/stages/stage_train/manifest/<train_config>_manifest.yaml` — манифест с путём к модели и идентификатором её MLflow run.

Отдельные выборки `oot` и `oos` в этом сценарии не используются.

3.8.4 Запуск этапа

Перед запуском необходимо выбрать конфигурации обучения и валидации в `params.yaml`. Входные данные и манифест формируются этапом `process`.

Для запуска из корня проекта:

```
dvc repro conf/stages/stage_train/dvc.yaml
```

4 Описание ноутбуков

Ноутбуки расположены в каталоге `notebooks/` и предназначены для пошагового разбора работы проекта, просмотра промежуточных результатов и проверки изменений. Они позволяют выполнять отдельные части этапа, проверять загруженные конфигурации и исследовать данные до сохранения результатов.

4.1 Параметры запуска

В начальных ячейках ноутбуков отдельными переменными заданы параметры запуска: имена конфигураций, входных манифестов и выходных артефактов. Далее эти значения передаются в Hydra через `compose` и список `overrides`.

Эти параметры соответствуют настройкам, которые при запуске через DVC передаются Python-модулям из команд этапов. Однако в ноутбуках значения задаются непосредственно в ячейках: изменение `params.yaml` не обновляет их автоматически.

Перед выполнением необходимо проверить выбранные конфигурации, входные данные и имена результатов. Ячейки следует выполнять последовательно: последующие шаги используют объекты, созданные на предыдущих.

В ноутбуках используются относительные пути от каталога `notebooks/`: корень проекта определяется как родительский каталог, а для обращения к конфигурациям и артефактам используется префикс ...

4.2 Загрузка данных: load.ipynb

Ноутбук `notebooks/load.ipynb` показывает работу этапа `load`. В начальной ячейке выбираются конфигурация источника и имя выгружаемой выборки.

Основные шаги:

1. Сборка конфигурации Hydra.
2. Чтение общих путей, параметров подключения и конфигурации источника.
3. Определение MLflow run и выгрузка таблицы из Teradata в CSV.
4. Сохранение манифеста загрузки и его передача в MLflow.
5. Просмотр первых строк полученного CSV.

Ноутбук позволяет проверить настройки источника и результат выгрузки отдельно от запуска через DVC.

4.3 Обработка данных: process.ipynb

Ноутбук `notebooks/process.ipynb` показывает работу этапа `process`. В начале задаются имена конфигураций исходных данных и преобразований, входного манифеста, обработанной выборки и состояния преобразований. Отдельно указывается режим `fit`.

После чтения исходных данных создаётся `MainTransformer`. В режиме `fit=true` преобразования обучаются, а их состояние сохраняется. В режиме `fit=false` используются параметры из ранее сохранённого состояния.

Результат преобразования сохраняется в CSV. Затем формируется описание числовых и категориальных признаков, создаётся манифест обработки и выполняется логирование артефактов в MLflow.

Ноутбук удобен для проверки отдельных преобразований, состава столбцов и состояния кодирования категорий.

4.4 Профилирование: profiler.ipynb

Ноутбук `notebooks/profiler.ipynb` показывает запуск проверок обработанных данных. В начальной ячейке задаются имя конфигурации профилирования и имя манифеста обработанной выборки.

Из манифеста читаются путь к данным, описание столбцов и идентификатор MLflow run. После загрузки CSV создаётся `MainProfiler`, который выполняет проверки из выбранной конфигурации.

Полученные отчёты сохраняются в каталог статистики и передаются в MLflow. Ноутбук позволяет отдельно проверить настройки профилирования и получаемые результаты.

4.5 Обучение: train.ipynb

Ноутбук `notebooks/train.ipynb` позволяет выполнять обучение по отдельным шагам. В начале выбираются конфигурации обучения и валидации, а также манифест обработанной обучающей выборки.

После загрузки конфигураций и данных формируются матрица признаков `X` и таргет `y`. Дальнейшая работа включает подбор гиперпараметров, подготовку финальной модели и её обучение.

Финальная модель получает фиксированные параметры и лучшие параметры найденного исследования Optuna. Для обучения используются `X_train` и `y_train`, а проверочная часть передаётся как `eval_set`. Обученная модель сохраняется в формате `joblib`.

Ноутбук предназначен для исследования процесса обучения: можно просматривать конфигурации и результаты поиска, а также отдельно выполнять финальное обучение. Подробное устройство компонентов обучения описано в разделе разработки.

4.6 Связь с основными этапами

Ноутбуки используют компоненты из `src/` и классы конфигураций проекта, но выполняют шаги вручную, по ячейкам. Они дополняют запуск через DVC, а не заменяют описание зависимостей между этапами.

Выполнение ячеек может создавать или перезаписывать CSV-файлы, состояния преобразований, модели и отчёты, а также записывать результаты в MLflow. Поэтому перед запуском необходимо проверять не только входные параметры, но и имена выходных артефактов.